

A
Project Review Meeting
on
LLM-Based Customer Support Chatbot with
Knowledge Retrieval

Submitted by

M.VENNELA 2420030218

R.PRAVALLIKA 2420090073

Group: 13 Section : 7

Specialization-

APPLIED MACHINE LEARNING FOR TEXT ANALYSIS

Under guidance of:
DR.K.SWANTHANA



Outline of the presentation

- Problem Statement
- Objectives
- Literature Survey
- Research Gap Identification
- Innovation
- Novelty
- Creativity
- Feasibility
- Project Plan

Problem Statement

- Traditional chatbots depend on rules and keywords, limiting their ability to handle complex queries.
- Large organizational knowledge bases make information retrieval difficult and time-consuming.
- Standalone LLMs may generate inaccurate or unsupported responses without organization-specific knowledge.
- Existing systems often lack context awareness and source transparency.
- Therefore, there is a need for a RAG-based intelligent chatbot that provides accurate, contextual, explainable, and reliable customer support.

Objectives

- **Develop** an intelligent customer-facing chatbot using an LLM for automated customer support.
- **Implement** a RAG system using **embeddings** and **FAISS** to retrieve relevant knowledge from support documents.
- **Generate** accurate and context-aware responses using the retrieved information.
- **Improve reliability and transparency** through **source attribution** and **confidence estimation**.
- **Support multi-turn conversations** using conversation history and contextual understanding.



Literature Survey

S. No.	Research Paper	Year	Method / Technology	Key Contribution	Relevance to Our Project	Link
1	Retrieval Enhancements for RAG: Insights from a Deployed Customer Support Chatbot — Daniel González Juclà et al.	2026	RAG, embeddings, chunking, RRF, LLM reranking	Studies retrieval improvements in a real deployed customer-support chatbot and shows that better retrieval improves downstream response quality.	Directly supports our embedding, chunking, FAISS retrieval and Top-K document selection.	ACL Anthology
2	Retrieval-Augmented Generation with Knowledge Graphs for Customer Service Question Answering — Zhentao Xu et al.	2024	RAG, LLM, Knowledge Graph, semantic retrieval	Uses structured knowledge from historical customer-service issues to improve retrieval and question answering; reports improved retrieval and generation performance.	Supports our customer-service QA and knowledge retrieval approach and provides a possible future Knowledge Graph extension.	Research Paper –



Literature Survey

S. No.	Research Paper	Year	Method / Technology	Key Contribution	Relevance to Our Project	Link
3	Supporting Human Operators During Customer Service Interactions with Agentic-RAG — Juan Barrionuevo-Valenzuela et al.	2026	Agentic-RAG, retrieval validation, summarization, LLM	Uses RAG to assist human customer-service operators with relevant information, summaries and suggested responses.	Directly supports our proposed human-agent escalation and conversation-context features.	ACL Anthology



Research Gap Identification

Paper	What the Paper Addresses	Identified Gap / Limitation	How Our Project Addresses the Gap
1. Juclà et al. (2026) — <i>Retrieval Enhancements for RAG: Insights from a Deployed Customer Support Chatbot</i>	Focuses mainly on improving RAG retrieval quality through embedding comparison, chunking, RRF, and LLM-based reranking in a deployed customer-support chatbot.	The study primarily focuses on retrieval optimization . There is scope to integrate retrieval with a broader customer-support system involving conversational context, explainability, confidence estimation, and intelligent escalation.	Our project combines semantic retrieval with LLM response generation, multi-turn conversation, source attribution, confidence estimation .
2. Xu et al. (2024) — <i>Retrieval-Augmented Generation with Knowledge Graphs for Customer Service Question Answering</i>	Uses RAG + Knowledge Graphs to preserve relationships within and between historical customer-service issues and improve retrieval and answer generation.	The approach relies on knowledge-graph construction from historical issue tickets , which adds structural complexity. There is scope for a more lightweight document-based RAG architecture suitable for broader customer-support knowledge such as FAQs, manuals and policies.	Our system uses FAISS to retrieve relevant information from diverse support documents, followed by LLM-based response generation.
3. Barrionuevo-Valenzuela et al. (2026) — <i>Supporting Human Operators During Customer Service Interactions with Agentic-RAG</i>	Develops an Agentic-RAG copilot that assists human operators through summaries, relevant information and suggested responses, using agents for routing, retrieval validation and response generation.	The primary focus is supporting human operators in call centers , rather than providing a complete self-service chatbot with user-facing explainability, confidence information and broader knowledge-base interaction.	Our project focuses on a direct customer-facing chatbot , while retaining human escalation for unresolved or low-confidence queries. It also proposes source attribution, confidence estimation and multi-turn conversation .

Innovation

- **RAG-Based Knowledge Retrieval:** Retrieves relevant information from FAQs, manuals, policies, and support documents before generating responses.
- **LLM-Powered Responses:** Uses an LLM such as Gemini or Llama 3 to generate natural, accurate, and context-aware answers.
- **Semantic Search:** Uses embeddings and **FAISS vector search** to understand the meaning of customer queries rather than relying only on keywords.
- **Explainable Responses:** Provides **source attribution and confidence estimation**, helping users understand and trust the generated answers.

Novelty

- **Unified AI Customer Support:** Integrates **RAG, LLM, semantic search, and conversational AI** into a single platform.
- **Knowledge-Grounded Answers:** Generates responses based on **retrieved organizational knowledge**, reducing unsupported answers.
- **Explainable Responses:** **confidence information** with the generated answer.

Creativity

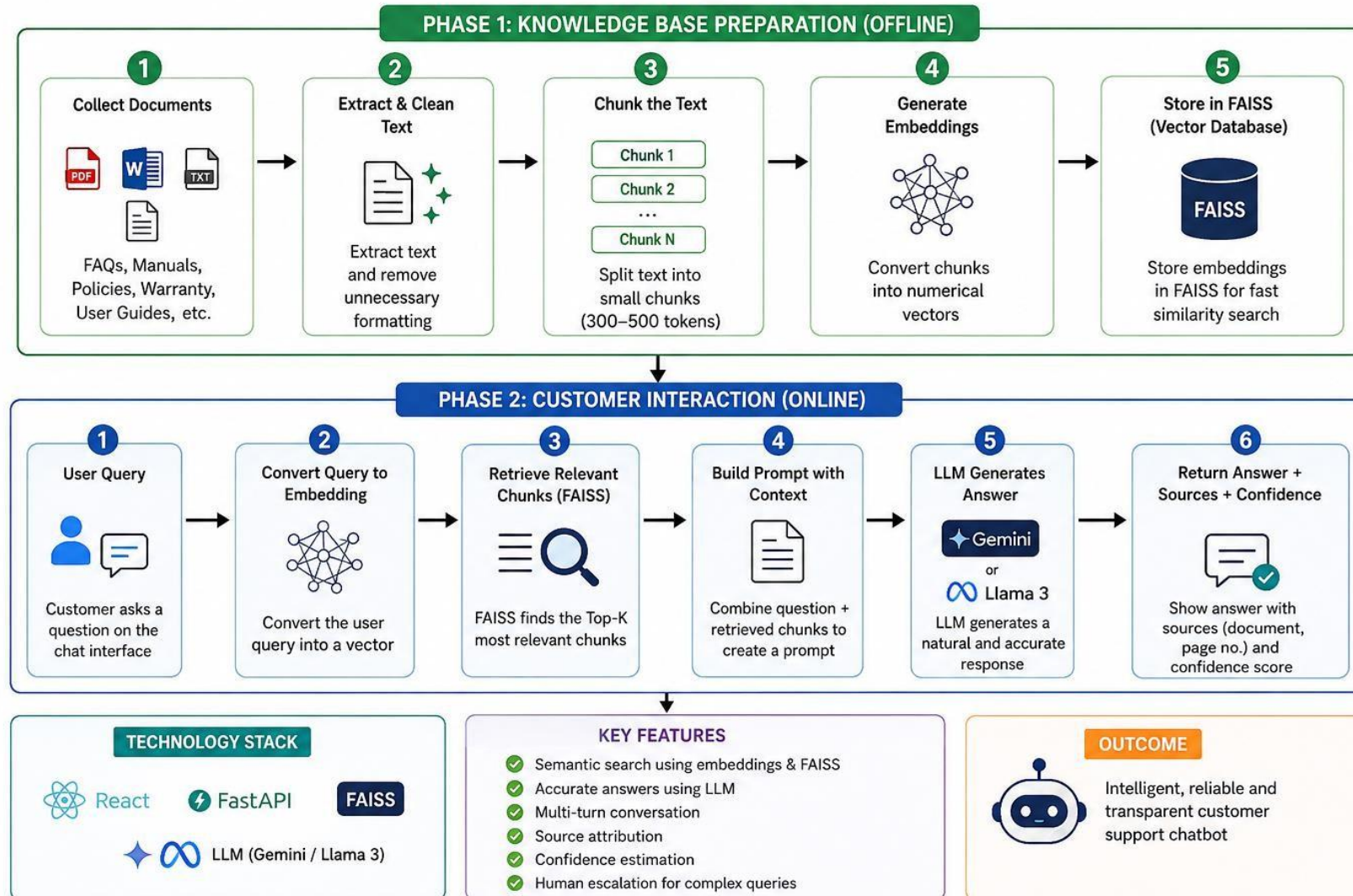
- **Natural Conversational Interaction:** Allows customers to ask questions in their own words instead of following predefined options.
- **Context-Aware Conversations:** Uses previous messages to understand follow-up questions and maintain conversation flow.
- **Conversation Summarization:** Summarizes previous interactions to help maintain context and support efficient human-agent handover.

Feasibility

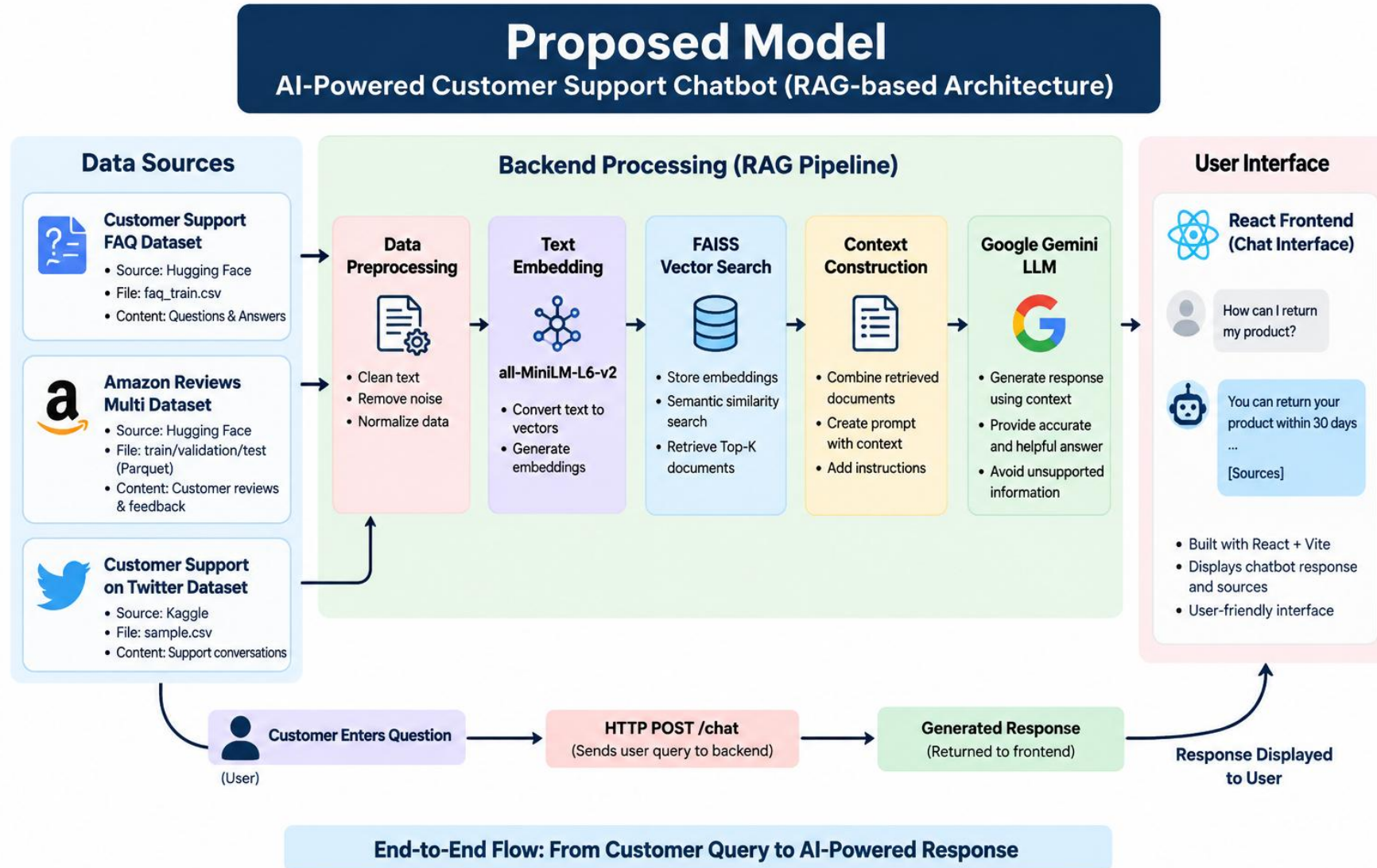
- **Technical Feasibility:** The system can be developed using existing technologies such as React, FastAPI, embeddings, FAISS, and LLMs such as Gemini or Llama 3.
- **Data Feasibility:** The chatbot can use available FAQs, product manuals, user guides, policies, and customer-support knowledge bases as its knowledge source.
- **Implementation Feasibility:** RAG allows the system to use existing LLMs without requiring the development or training of a large language model from scratch.
- **Scalability:** The knowledge base can be expanded by adding new documents and updating their embeddings.
- **Evaluation Feasibility:** The system can be evaluated using retrieval accuracy, response relevance, answer correctness, semantic similarity, response latency, and user satisfaction.

Project Plan

LLM-Based Intelligent Customer Support Chatbot using RAG



Proposed Model



Combining real customer data with semantic search and generative AI to deliver accurate and helpful support.

Project Implementation

1. Data Preparation

Collect customer-support FAQs → preprocess and clean the text → generate **Transformer-based embeddings** using **all-MiniLM-L6-v2** → store them in a **FAISS vector index**.

2. Product Selection & Query

Customer selects their **purchased product** and enters a support question.

3. Product-Aware Retrieval

The question is converted into an embedding → relevant product-specific information is filtered and retrieved using **FAISS Top-K semantic search**.

4. Prompt Construction

The retrieved FAQ information is combined with the customer's question to create a **context-aware prompt**.

5. Response Generation

The prompt is sent to **Gemini LLM**, which generates a natural, knowledge-grounded response.

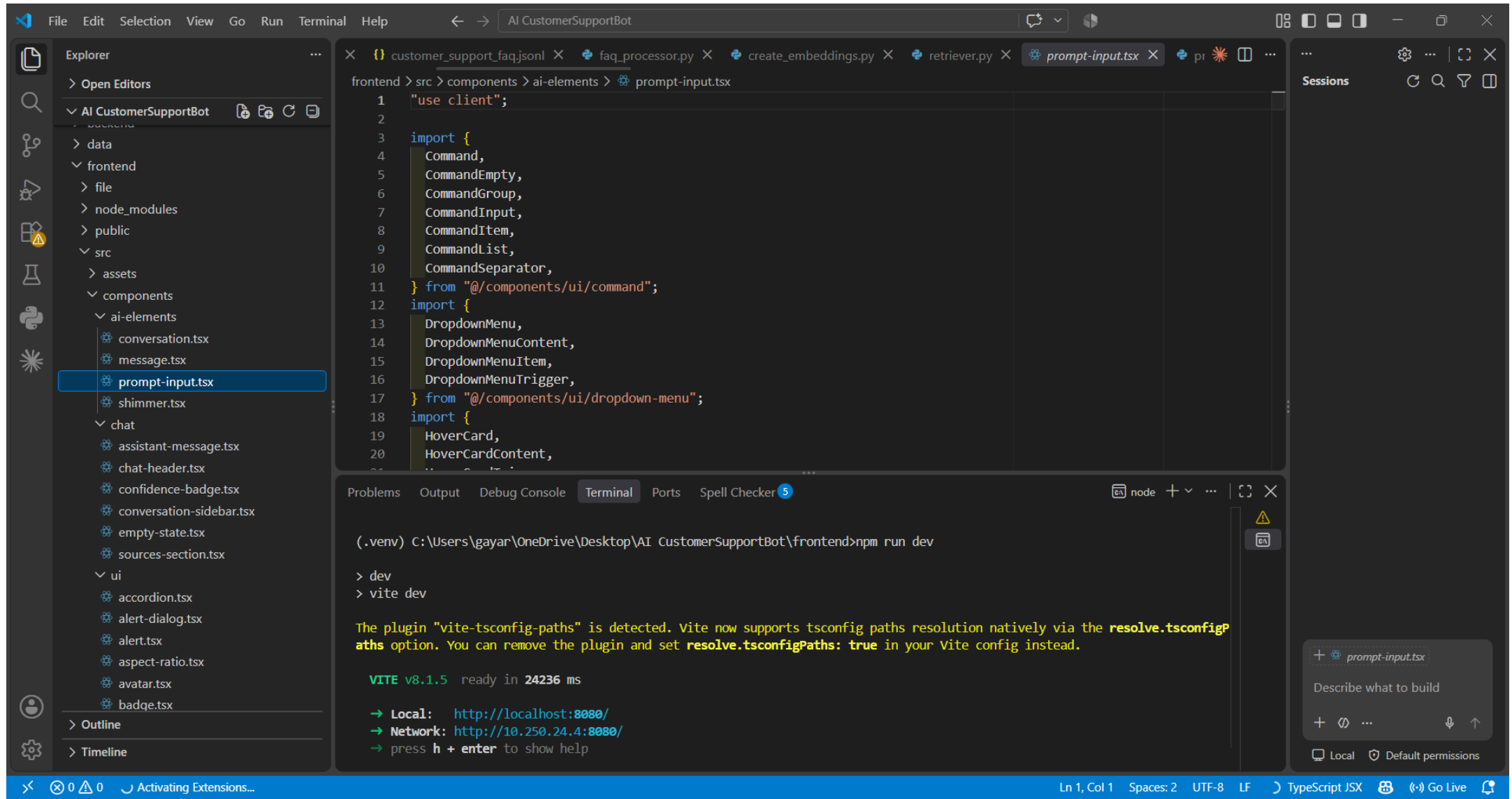
6. User Interaction

The response and relevant sources are returned through the **FastAPI backend** and displayed in the **React chat interface**.

Overall Flow

Customer → Product Selection → Query Embedding → Product-Aware FAISS Retrieval → Context + Prompt → Gemini → Grounded Response → React UI

Frontend Code



VS Code interface showing the frontend code for the AI CustomerSupportBot project.

Explorer: The file explorer on the left shows the project structure. The selected file is `prompt-input.tsx` located in the `src > components > ai-elements` directory.

Code Editor: The main editor displays the content of `prompt-input.tsx`. The code includes imports for various components from the `@/components/ui` directory.

```
1 "use client";
2
3 import {
4   Command,
5   CommandEmpty,
6   CommandGroup,
7   CommandInput,
8   CommandItem,
9   CommandList,
10  CommandSeparator,
11 } from "@/components/ui/command";
12 import {
13   DropdownMenu,
14   DropdownMenuContent,
15   DropdownMenuItem,
16   DropdownMenuTrigger,
17 } from "@/components/ui/dropdown-menu";
18 import {
19   HoverCard,
20   HoverCardContent,
```

Terminal: The terminal at the bottom shows the command `npm run dev` being executed. The output indicates that the development server is running on `http://localhost:8080/`.

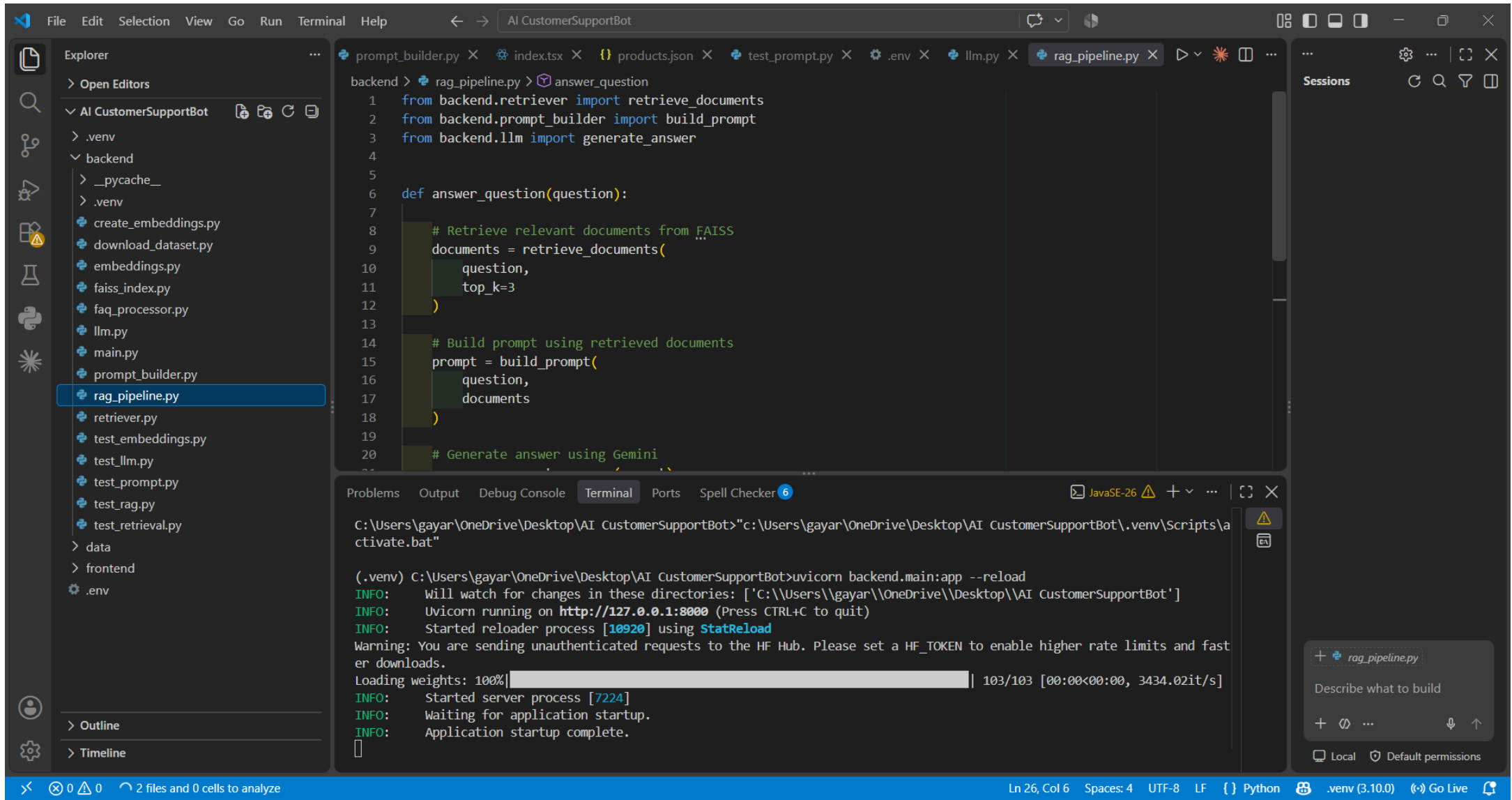
```
(.venv) C:\Users\lgayar\OneDrive\Desktop\AI CustomerSupportBot\frontend>npm run dev
> dev
> vite dev

The plugin "vite-tsconfig-paths" is detected. Vite now supports tsconfig paths resolution natively via the resolve.tsconfigPaths option. You can remove the plugin and set resolve.tsconfigPaths: true in your Vite config instead.

VITE v8.1.5 ready in 24236 ms
→ Local: http://localhost:8080/
→ Network: http://10.250.24.4:8080/
→ press h + enter to show help
```

Bottom Bar: The status bar at the bottom shows the current file is `prompt-input.tsx`, the encoding is `UTF-8`, and the language is `TypeScript JSX`.

Backend Code



VS Code interface showing the backend code for an AI CustomerSupportBot.

Explorer:

- AI CustomerSupportBot
 - .env
 - backend
 - __pycache__
 - .env
 - create_embeddings.py
 - download_dataset.py
 - embeddings.py
 - faiss_index.py
 - faq_processor.py
 - llm.py
 - main.py
 - prompt_builder.py
 - rag_pipeline.py**
 - retriever.py
 - test_embeddings.py
 - test_llm.py
 - test_prompt.py
 - test_rag.py
 - test_retrieval.py
 - data
 - frontend
 - .env

rag_pipeline.py:

```
1 from backend.retriever import retrieve_documents
2 from backend.prompt_builder import build_prompt
3 from backend.llm import generate_answer
4
5
6 def answer_question(question):
7
8     # Retrieve relevant documents from FAISS
9     documents = retrieve_documents(
10         question,
11         top_k=3
12     )
13
14     # Build prompt using retrieved documents
15     prompt = build_prompt(
16         question,
17         documents
18     )
19
20     # Generate answer using Gemini
```

Terminal:

```
C:\Users\gayar\OneDrive\Desktop\AI CustomerSupportBot>"c:\Users\gayar\OneDrive\Desktop\AI CustomerSupportBot\.venv\Scripts\activate.bat"
(.venv) C:\Users\gayar\OneDrive\Desktop\AI CustomerSupportBot>uvicorn backend.main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\gayar\\OneDrive\\Desktop\\AI CustomerSupportBot']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [10920] using StatReload
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100% | 103/103 [00:00<00:00, 3434.02it/s]
INFO: Started server process [7224]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Bottom Bar: Ln 26, Col 6 Spaces: 4 UTF-8 LF Python .venv (3.10.0) Go Live

OUTPUT

CONVERSATIONS

+ New Chat

How can I return a product?



AI Customer Support

Knowledge-Based Support Assistant

AI Assistant Online

How can I return a product?

08:57 AM

ANSWER

To return a product, please ensure that it is in its original condition. While having the original packaging and receipt is preferred, you can still make a return without them.

For instructions on how to process your return, please refer to our return policy. If you are missing your receipt or original packaging, or need further guidance, please contact our customer support team and we will be happy to assist you!

SOURCES

Sources (3)

08:57 AM

Answers are grounded in your knowledge base via the FastAPI backend.

Ask a question about our products, policies, warranty, shipping or returns...

Enter to send · Shift + Enter for a new line



Output

CONVERSATIONS

+ New Chat

How long does shipping take?

AI Customer Support
Knowledge-Based Support Assistant

AI Assistant Online

How long does shipping take?

10:49 AM

ANSWER

Shipping times depend on your destination and the shipping method you choose:

***Standard shipping:** Usually takes 3-5 business days.

***Express shipping:** Usually takes 1-2 business days.

Let me know if you have any other questions!

SOURCES

Sources (3)

10:50 AM

Ask a question about our products, policies, warranty, shipping or returns...

Enter to send · Shift + Enter for a new line

Answers are grounded in your knowledge base via the FastAPI backend.